Beginner's Guide: Fetching Data in React with Node.js API

1. Setting up the Node.js Backend

1.1 Initialize the project

First, we need to set up our Node.js project:

```
npm init
```

This command creates a `package.json` file that will manage our project's dependencies.

1.2 Install Express

Express is a popular web framework for Node.js. Install it with:

```
npm install express
```

1.3 Create sample data (data.js)

```
module.exports = [
    { id: 1, name: "Item 1" },
    { id: 2, name: "Item 2" },
    { id: 3, name: "Item 3" }
];
```

1.4 Create the API endpoint (app.js)

```
const express = require('express');
const data = require('./data');
```

```
const app = express();


app.get('/api/sampleData', (req, res, next) => {
    res.send(data);
});


app.listen(3000, () => {
    console.log(`Server Started`);
});
```

What this code does:

- We import Express and our data.

- We create an Express application.

- We define a route `/api/sampleData` that sends our data when requested.

- We start the server on port 3000.

2. Setting up the React Frontend

2.1 Create a React component (DataComponent.jsx)

```
import { useState, useEffect } from "react";


const API_endpoint = 'http://localhost:3000/api/sampleData'
```

```
export default function DataComponent() {
    const [data, setData] = useState([]);

    useEffect(() => {
        async function fetchPosts() {
            const response = await fetch(`${API_endpoint}`);
            const responseData = await response.json();
            setData(responseData);
        }
        fetchPosts();
    }, [])

    return (
        <section>
            {data.map((item) => (
                <div key={item.id}>{item.name}</div>
            ))}
        </section>
    );
}
```

What this code does:

- We import `useState` and `useEffect` hooks from React.

- We define the API endpoint URL.

- We create a state variable `data` to store our fetched data.

- We use `useEffect` to fetch data when the component mounts.

- In the `fetchPosts` function, we make a GET request to our API and update the state with the response.

- We render the fetched data in the component's return statement.

3. Handling CORS Issues

3.1 Install the CORS package

```
npm install --save cors
```

3.2 Update app.js to use CORS

Add these lines to your `app.js`:

```
const cors = require('cors');

app.use(cors());
```

Place these lines after the `const app = express();` line and before defining any routes.

4. Running the Application

1. Start your Node.js server:

```
node app.js
```

You should see "Server Started" in the console.

2. Start your React application (assuming you're using Create React App):

```
npm start
```

Now, your React application should be able to fetch and display data from your Node.js API!

Conclusion

This guide demonstrates how to create a simple API with Node.js and Express, and how to fetch and display that data in a React application. Remember to handle errors and add more complex functionality as needed in a real-world application.